PYNTREE TECHNOLOGIES

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

2 MONTHS

An INTERNSHIP REPORT

Submitted by

RENUKA S

In partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SSM INSTITUTE OF ENGINEERING AND TECHNOLOGY

DINDIGUL 624 002

OCTOBER 2026

BONAFIDE CERTIFICATE

This is to certify that RENUKA S (922123243043), is a bonafide student of AI&DS Department, SSM Institute of Engineering and Technology, Dindigul - 624 002, affiliated to Anna University.

The student is studying in VII Semester during the academic year 2026-2027 and has undertaken the Industry Internship at Pyntree Technologies from 01/07/2026 to 01/09/2026 as part of the academic requirements of the programme.

SIGNATURE	**SIGNATURE**
Dr.M.PREMKUMAR **HEAD OF THE DEPARTMENT** **Department of Artificial Intelligence and Data Science,** **SSM Institute of Engineering and Technology,** **Dindigul - 624002.**	**Ms. I. JOICY** **ASSISTANT PROFESSOR** **Department of Artificial Intelligence and Data Science,** **SSM Institute of Engineering and Technology,** **Dindigul - 624002.**

OFFER LETTER

PYNTREE TECHNOLOGIES

1st Floor, SMA Complex, Mendonsa Colony, Nagal Nagar, Dindigul, Tamil Nadu - 624003
Website: www.pyntree.com | Email: hr@pyntree.com

Ref No: PT/INT/2026/AIML-042 **Date: July 1, 2026**

OFFER OF INTERNSHIP

Dear Renuka S,

Following your application and recent technical discussion, Pyntree Technologies is pleased to offer you an internship position as an **Artificial Intelligence & Machine Learning (AI/ML) Intern** at our office in Dindigul, Tamil Nadu. We were impressed by your academic background in Artificial Intelligence & Data Science and look forward to your contributions.

1. INTERNSHIP ENGAGEMENT DETAILS

This internship will be based in the AI & Data Science Department at our Dindigul office for a duration of 3 months (extendable based on performance evaluation). Your commencement date will be mutually agreed upon, and you will report directly to Mr. Karthikeyan (Technical Director / Lead). If required for specific project requirements, training modules, or official company purposes, you may be requested to visit or temporarily report to our Chennai branch.

2. ROLES AND RESPONSIBILITIES

As an AI/ML Intern, your core duties include data cleaning, preprocessing, and exploratory data analysis; building, fine-tuning, and evaluating machine learning and deep learning models using Python frameworks such as PyTorch, TensorFlow, and Scikit-Learn; feature engineering and hyperparameter optimization; collaborating with developers to integrate models into web applications and REST API microservices; and maintaining version-controlled codebases on Git/GitHub with weekly progress reports.

3. TERMS & CODE OF CONDUCT

During your tenure, you must maintain strict confidentiality regarding all proprietary source code, client datasets, and internal technical specifications. All software, algorithms, and documentation created remain the exclusive intellectual property of Pyntree Technologies. Professional decorum and schedule adherence are mandatory. Either party may terminate this agreement with a 7-day written notice. An official Internship Completion Certificate will be issued upon successful completion of your project deliverables.

Please review the terms above and return a signed copy within 5 business days from the date of issue to confirm your acceptance. We look forward to working with you at Pyntree Technologies.

For Pyntree Technologies,



Karthikeyan
Authorized Signatory / Technical Director
Pyntree Technologies, Dindigul

Candidate Acceptance:

Name: Renuka S
Date: July 1, 2026

ACKNOWLEDGEMENT

First and foremost, we would like to express our deep sense of gratitude to our most honorable Chairman and Management Trustees **Mr. K. Shanmugavel** for providing us with necessary facilities during the course of study. We feel immensely pleased to express our sincere thanks to our Principal

Dr. S. Karthigai Lakshmi for encouragement and support extended by them. We extend our solemn gratitude to **Dr. M. Premkumar**, Head of the Department, Artificial Intelligence and Data Science, for timely support to all our activities.

I would like to thank my Faculty Internship Coordinator, the Faculty In-Charge, for the continuous guidance, valuable suggestions, and encouragement during the internship and in the preparation of this report.

I am grateful to Pyntree Technologies for providing me with the opportunity to gain practical exposure to an industry environment. I sincerely thank my Industry Mentor, Karthikeyan, Software Developer, and the entire team for their guidance, technical support, and valuable insights throughout the internship.

We express our sincere thanks to our guide **Ms. I. Joicy, AP/AI&DS** for her guidance throughout our Internship. Also we express our thanks to the faculty members of our department, non-teaching staff members and my dear friends for their moral support, help and encouragement towards the successful completion of the internship.

INDEX

S. No.	CONTENTS	PAGE NO.
1	Bonafide Certificate	2
2	Offer Letter	3
3	Acknowledgement	4
4	Executive Summary	8
5	List of Figures	9
6	List of Tables	9
7	Abbreviations / Acronyms	9
8	**Chapter 1 – Introduction**	10
8.1	Background	10
8.2	Internship Overview	10
8.3	Need and Purpose of the Internship	11
8.4	Objectives of the Internship	11
8.5	Scope of the Report	11
9	**Chapter 2 – Company Profile**	12
9.1	Company Introduction	12
9.2	Vision and Mission	12
9.3	Products / Services	12
9.4	Industry Domain	12
9.5	Department / Team Assigned	12
10	**Chapter 3 – Internship Activities**	13
10.1	Internship Plan	13
10.2	Training Received	13
10.3	Tasks and Responsibilities	13
10.4	Technologies and Tools Used	14
10.5	Weekly / Phase-wise Activities	14
10.6	Team Collaboration	14
11	**Chapter 4 – Major Project / Work Undertaken**	15

11.1	Project Title	15
11.2	Problem Statement	15
11.3	Objectives	15
11.4	Requirements	15
11.5	Existing System / Approach	15
11.6	Proposed Solution	15
11.7	Methodology	15
11.8	System Architecture / Workflow	16
11.9	Modules / Components	17
11.10	Implementation	17
11.11	Results and Outputs	17
11.12	Screenshots / Demonstration Evidence	18
11.13	Additional Project for Reference: LearnLock	18
11.14	Testing and Validation Approach	21
12	**Chapter 5 – Skills and Learning Outcomes**	22
12.1	Technical Skills	22
12.2	Software / Tools / Platforms	22
12.3	AI / Cloud / Data Skills	22
12.4	Communication and Teamwork	22
12.5	Problem-Solving Skills	22
12.6	Professional Learning	22
13	**Chapter 6 – Challenges and Solutions**	23
13.1	Technical Challenges	23
13.2	Organizational / Team Challenges	23
13.3	Solutions Adopted	23
13.4	Key Lessons Learned	23
14	**Chapter 7 – Professional Experience**	24
14.1	Workplace Practices	24
14.2	Industry Standards	24

14.3	Professional Ethics	24
14.4	Safety, Security and Data Practices	24
15	**Chapter 8 – Conclusion and Future Scope**	25
15.1	Conclusion	25
15.2	Achievement of Objectives	25
15.3	Career Relevance	25
15.4	Future Scope / Recommendations	25
16	References	26
17	Appendices	27
17.1	Appendix A – Internship Certificate	27
17.2	Appendix B – Internship Offer / Permission Letter	27
17.3	Appendix C – Weekly Internship Logbook	27
17.4	Appendix D – Industry Mentor Evaluation	28
17.5	Appendix E – Final Internship Presentation / Viva Record	28
17.6	Appendix F – Photographs, Screenshots and Project Evidence	28
18	Completion Certificate	30

Executive Summary

This report documents a 2 months internship in the domain of AI/ML carried out at Pyntree Technologies, Dindigul, from 01 July 2026 to 01 September 2026. The internship was oriented towards applied AI and machine learning: building, fine-tuning and evaluating models, data preprocessing and analysis, and integrating models into web applications and REST API microservices, with all work kept under version control. The practical work centred on a RAG Chatbot - An AI Knowledge Assistant. The project focuses on building a retrieval-augmented generation (RAG) chatbot that answers user questions grounded in an organisation's own documents, pairing semantic retrieval over a vector store with a large language model so that answers stay accurate, current and traceable to their sources. The report describes the layered architecture designed for the platform, the lifecycle of a query from the user's question through retrieval to a grounded answer, the functional modules involved, and the individual contribution made to them. It closes with the engineering practices absorbed during the internship - version control, review and testing - and the learning outcomes drawn from them. An additional project, LearnLock - an Android application that blocks selected apps until a multiple-choice question is answered correctly - is documented in Chapter 4 as a reference project, showing the same layered engineering practice in a mobile setting.

LIST OF FIGURES

FIGURE NO	FIGURE	PAGE NO
Figure 1	System architecture of the RAG chatbot	16
Figure 2	Retrieval-augmented query lifecycle	17
Figure 3	Mind map of internship activities and learning	18
Figure 4	System architecture of LearnLock	20
Figure 5	LearnLock lock, answer and unlock lifecycle	20

LIST OF TABLES

TABLE NO	TABLE	PAGE NO
Table 1	Layers of the LearnLock application	19
Table 2	Weekly internship logbook	27
Table 3	Areas of guidance from the industry mentor	28

ABBREVIATIONS / ACRONYMS

AI - Artificial Intelligence
ML - Machine Learning
RAG - Retrieval-Augmented Generation
API - Application Programming Interface
LLM - Large Language Model
APK - Android Package Kit
MCQ - Multiple-Choice Question
JSON - JavaScript Object Notation

CHAPTER 1
INTRODUCTION

Classroom study of artificial intelligence usually stops at the model: a dataset is cleaned, an algorithm is trained, and an accuracy figure is reported. Industry practice begins where that exercise ends. A model earns its value only once it is wrapped in a service, fed by a reliable data path, deployed behind an interface and maintained as requirements change. This internship was undertaken to work on that second half of the problem, applied to a language-model application.

1.1 BACKGROUND

Classroom study of artificial intelligence usually stops at the model: a dataset is cleaned, an algorithm is trained, and an accuracy figure is reported. Industry practice begins where that exercise ends. A model earns its value only once it is wrapped in a service, fed by a reliable data path, deployed behind an interface and maintained as requirements change. This internship was undertaken to work on that second half of the problem, applied to a language-model application.

The need for this second half is felt most strongly in applications built on language models. A user does not see an embedding or a vector store; the user sees an answer, and the only question that matters is whether that answer is correct and can be trusted. Every stage behind it, from document preparation to retrieval and prompt assembly, has to be correct for the answer to be worth reading.

The internship was undertaken at Pyntree Technologies as an AI/ML Intern for a period of 2 Months (July - September 2026). The departmental review meeting held on 17.08.2026 collected and verified the internship details, the nature of the AI/ML and software-development work, and the project involvement.

The weeks that followed were spent inside a real engineering workflow. Requirements were discussed with the industry mentor, implemented on a branch, tested locally and reviewed before they were merged, and the same short cycle was followed for data preparation, model experiments and service integration. The resulting experience is the subject of this report.

1.2 INTERNSHIP OVERVIEW

The internship ran for 2 Months from 01 July 2026 to 01 September 2026 in the AI/ML domain, with work carried out from Dindigul, Tamil Nadu / Remote. The role was that of an AI software developer contributing to a live product, RAG Chatbot - An AI Knowledge Assistant.

Working from Dindigul and remotely, the intern took part in incremental delivery and reported progress to the industry mentor, Karthikeyan, Software Developer. The RAG chatbot served as the common thread through the two months: it supplied the problem, the architecture to be understood and the codebase in which changes had to be made carefully.

1.3 NEED AND PURPOSE OF THE INTERNSHIP

The internship connected academic study with practical engineering tasks: building, fine-tuning and evaluating models, preparing data, and integrating models into web applications and REST API microservices under version control.

Three needs shaped the internship. The first was practical exposure: moving from curated coursework datasets to real source documents that are inconsistent in format and quality. The second was integration: learning how a model, a retrieval service and an interface are connected so that a result actually reaches the user. The third was professional discipline: version control, code review, testing and respect for confidentiality, none of which can be learned from a textbook alone.

1.4 OBJECTIVES OF THE INTERNSHIP

Gain hands-on experience in building, fine-tuning and evaluating machine learning and deep learning models. Work through a complete data path: cleaning, preprocessing, exploratory analysis, feature engineering and hyperparameter optimisation. Develop models with Python frameworks - PyTorch, TensorFlow and Scikit-Learn. Integrate models into web applications and REST API microservices under version control with Git and GitHub. Contribute to a retrieval-augmented generation (RAG) chatbot as the practical outcome of the internship.

Each objective was tied to something that could be observed at the end of the internship: a repeatable ingestion path for documents, a retrieval step whose quality could be compared between changes, grounded answers that cite their sources, and a history of reviewed commits on Git and GitHub.

1.5 SCOPE OF THE REPORT

The report covers the internship engagement and its terms, the areas of training, the architecture and functional design of the RAG chatbot, the individual contribution, and the learning outcomes. Architecture and design are discussed at a level appropriate for an academic report; proprietary source code, model weights and internal datasets are deliberately excluded.

The chapters that follow are arranged in the order recommended by the department. Chapter 2 introduces the company, Chapter 3 the internship activities, Chapter 4 the project undertaken, Chapter 5 the skills gained, Chapter 6 the challenges faced, Chapter 7 the professional experience, and Chapter 8 the conclusion and future scope. The internship offer letter, the references and the supporting appendices complete the report.

CHAPTER 2
COMPANY PROFILE

2.1 COMPANY INTRODUCTION

Pyntree Technologies is a technology company located at 1st Floor, SMA Complex, Mendonsa Colony, Nagal Nagar, Dindigul, Tamil Nadu - 624003. The internship placed the student with the engineering team as an AI software developer, contributing to live product work in artificial intelligence and machine learning rather than to separate training exercises.

The company's working model is project based. Work arrives as product requirements, is divided into small units, and is delivered through a shared version-controlled repository in which every change is reviewed. Interns join this model from the first week instead of being kept in a separate training track, so the learning is gained from the same discipline that the engineering team itself follows.

2.2 VISION AND MISSION

As recorded in the internship report, the company engages interns in live product work in artificial intelligence and machine learning with the engineering team, rather than separate training exercises.

2.3 PRODUCTS / SERVICES

The project undertaken during the internship was RAG Chatbot - An AI Knowledge Assistant. It answers user questions grounded in an organisation's own documents, pairing semantic retrieval over a vector store with a large language model so that answers stay accurate, current and traceable to their sources. It is discussed in Chapter 4. LearnLock, an AI-assisted Android study-lock application, is documented alongside it as an additional project for reference.

2.4 INDUSTRY DOMAIN

The internship domain was Artificial Intelligence and Machine Learning: building, fine-tuning and evaluating models, data preprocessing and analysis, and integrating models into web applications and REST API microservices.

2.5 DEPARTMENT / TEAM ASSIGNED

The student worked as an AI software developer with the engineering team under the guidance of Karthikeyan, Software Developer, contributing to RAG Chatbot - An AI Knowledge Assistant.

CHAPTER 3
INTERNSHIP ACTIVITIES

3.1 INTERNSHIP PLAN

The internship ran for 2 Months from 01 July 2026 to 01 September 2026 as AI/ML Intern at Pyntree Technologies, organised incrementally with each unit beginning with requirement discussion, followed by implementation, review and integration, with every change committed on Git and GitHub and reviewed before merge.

In the first weeks the plan concentrated on orientation: reading the existing code, setting up the development environment and learning the branching and review conventions on Git and GitHub. The later weeks moved to retrieval, generation and integration, with testing and refinement of answer quality in response to review comments.

3.2 TRAINING RECEIVED

Training spanned five connected areas: AI/ML Development with hands-on experience building, fine-tuning, and evaluating machine learning and deep learning models; Data Preprocessing and Analysis including cleaning, exploratory analysis, feature engineering, and hyperparameter optimization; Python Frameworks of PyTorch, TensorFlow, and Scikit-Learn; Integration and Version Control into web applications and REST API microservices with Git/GitHub; and Practical Learning applied through the RAG chatbot.

Training took place on the job rather than in a classroom. The industry mentor explained the architecture of the chatbot, walked through representative changes, and gave feedback on the intern's own commits, so that each area of training was practised on real tasks and then corrected through review.

3.3 TASKS AND RESPONSIBILITIES

The internship work covered AI/ML development and the data and integration tasks of the RAG chatbot: document loading, cleaning and chunking; embedding and indexing into the vector store with semantic retrieval; grounded answer generation with citations; data preprocessing and analysis; model building, fine-tuning and evaluation with PyTorch, TensorFlow and Scikit-Learn; REST API integration; and version control discipline on Git and GitHub.

Each task was handled as a small, reviewable unit. A typical task began with a discussion of what the change should do, continued with an implementation on a separate branch and focused tests, and ended with a review in which the mentor's comments were applied before the change was merged. Keeping tasks small made it easier to find the cause of a problem and to keep unfinished work out of the product.

Where a task involved documents or data, an additional step was added before implementation: inspecting the material that would feed the work, noting missing

or inconsistent content, and agreeing with the mentor on how each case should be handled. This avoided building a feature on assumptions about data that later proved to be wrong.

3.4 TECHNOLOGIES AND TOOLS USED

Python, PyTorch, TensorFlow, Scikit-Learn, data preprocessing, model evaluation, REST API integration, and Git/GitHub, together with a vector store for embeddings, an embedding model, a large language model, and a document store. For the additional LearnLock project: Flutter and Dart with Riverpod, go_router, sqflite and dio, Kotlin for native Android enforcement, and a FastAPI backend deployed on Cloud Run.

Version control was used for every change, with commits that were small, focused and described in plain language. Service endpoints were exercised through requests and responses during testing, and experiments with chunk size, embeddings and ranking were recorded so that a result could be reproduced later by rerunning the same steps.

3.5 WEEKLY / PHASE-WISE ACTIVITIES

Month 1 - Foundations, data and models: understanding the product and codebase, environment and Git/GitHub setup, data cleaning, preprocessing and exploratory analysis, and model building, fine-tuning and evaluation. Month 2 - Retrieval, generation and integration: document chunking, embeddings and semantic retrieval; grounded answer generation with citations, REST API integration, testing and improvement of retrieval and answer quality.

Across the two months the working rhythm stayed the same: a short planning discussion at the start of each unit of work, implementation and testing through it, and a review of what had been completed at the end. Progress against each stage of the retrieval pipeline was recorded in the logbook that appears in Appendix C.

3.6 TEAM COLLABORATION

Work was carried out with developer guidance and project feedback from Karthikeyan, Software Developer, with version control on Git and GitHub and review before merge.

Collaboration took several forms: asking precise questions when a requirement was unclear, explaining the intent of a change when it was submitted for review, and reading the comments of the reviewer carefully before replying. This improved both the quality of the intern's contributions and the speed with which they were accepted.

CHAPTER 4
MAJOR PROJECT / WORK UNDERTAKEN

4.1 PROJECT TITLE

RAG Chatbot - An AI Knowledge Assistant.

4.2 PROBLEM STATEMENT

A large language model on its own answers from what it learned during training. That creates two problems for any organisation that wants to use one over its own material. The model does not know private or recent documents, so it cannot answer questions about them; and when it does not know, it tends to produce a fluent answer anyway, with no way for the reader to check it. For factual question answering, both are disqualifying.

4.3 OBJECTIVES

The project focuses on building a retrieval-augmented generation (RAG) chatbot that answers user questions grounded in an organisation's own documents, pairing semantic retrieval over a vector store with a large language model so that answers stay accurate, current and traceable to their sources.

4.4 REQUIREMENTS

Source documents with cleaning and passage splitting; an embedding model for passages and queries; a vector store for semantic search with reranking; a large language model for grounded generation with citations; a chat interface keeping conversation context; an administrative view for the document collection; and logging with feedback capture for quality evaluation.

4.5 EXISTING SYSTEM / APPROACH

The existing approach is a language model answering from training memory alone, which cannot use private or recent documents and produces fluent but unverifiable answers when it does not know.

4.6 PROPOSED SOLUTION

Retrieval-augmented generation separates knowledge from reasoning. Documents are split into passages, converted to embeddings and stored in a vector store. When a question arrives, the most relevant passages are retrieved and placed before the language model as context, and the model answers from that context and cites it, grounding the answer in real documents.

4.7 METHODOLOGY

Work was organised incrementally: requirement discussion, implementation, review and integration, with Git and GitHub commits reviewed before merge. The pipeline ran from document preparation through embeddings and retrieval to grounded generation and REST API integration, with testing and quality improvement.

4.8 SYSTEM ARCHITECTURE / WORKFLOW

The platform is organised into four layers. The client layer holds the chat interface with citations, the admin console for the document collection, and API consumers. The application and API layer uses a gateway for authentication, rate limiting, session handling and routing, with a chat orchestrator, ingestion, retrieval, generation, and feedback and logging services. The RAG pipeline layer holds the document loader with cleaning and chunking, the embedding model, the semantic retriever with reranking, and the language model with prompt assembly joining query and context. The data and model infrastructure layer holds the vector store, the document store with metadata, and the served language and embedding models, with version control and continuous integration as the delivery path.

Figure 1: System Architecture of the RAG Chatbot

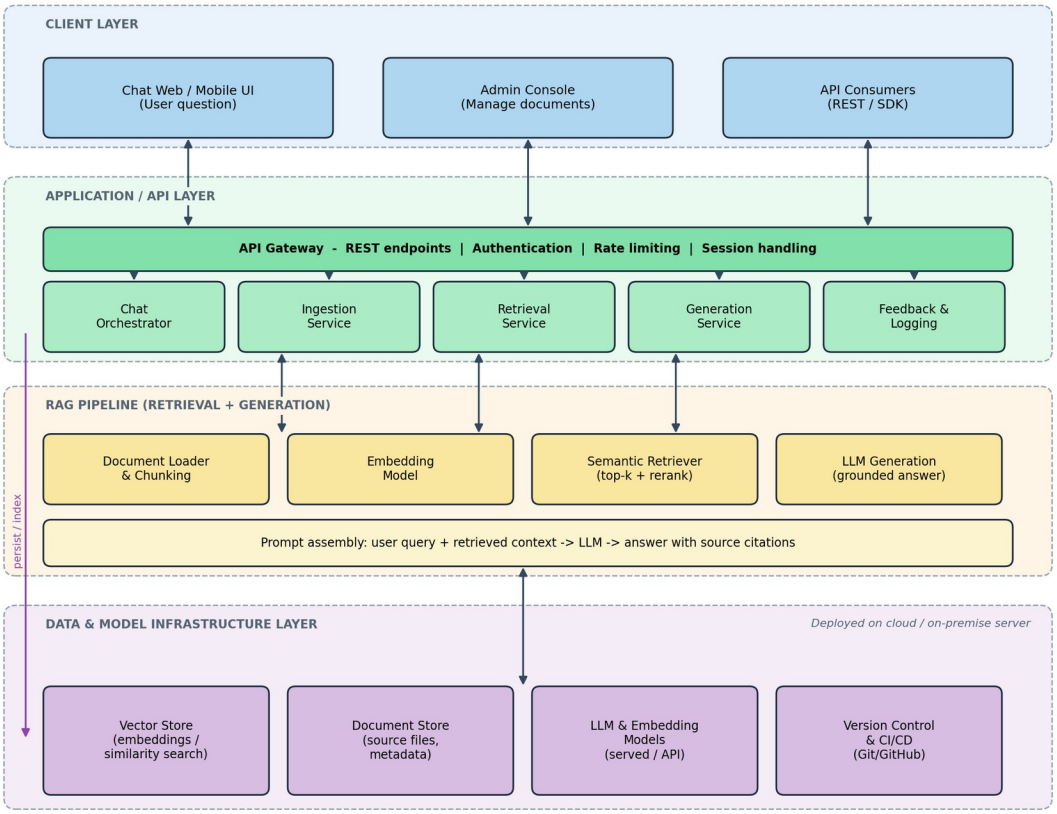


Figure 1: System architecture of the RAG chatbot

A single question is embedded and matched against the vector store, the closest passages are retrieved and reranked, and only then is the language model asked to answer from that context with citations. Without the retrieved passages the model would answer from memory and traceability would be lost.

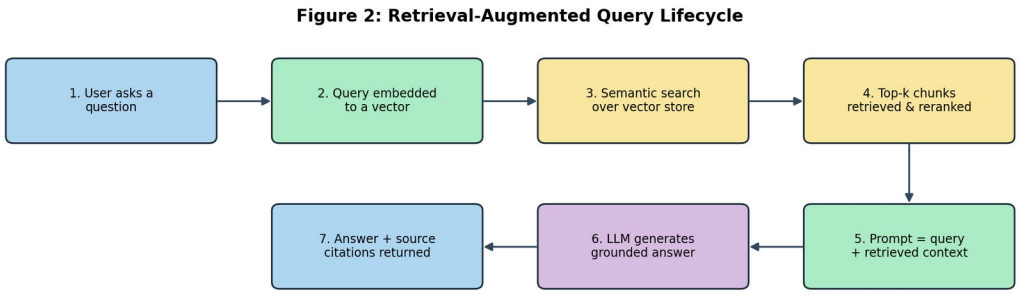


Figure 2: Retrieval-augmented query lifecycle

4.9 MODULES / COMPONENTS

Chat orchestrator coordinates a request across retrieval and generation and keeps conversation context. Ingestion service loads, cleans and chunks source documents. Embedding model converts passages and queries into vectors. Retrieval service finds and reranks the most relevant passages. Generation service prompts the language model to answer from retrieved context and cite sources. Vector store serves semantic similarity search. Document store holds source files and metadata. Feedback and logging records queries, answers and feedback for evaluation.

4.10 IMPLEMENTATION

Implementation covered document ingestion with loading, cleaning and chunking; embedding and indexing with semantic retrieval; prompt assembly for grounded generation with citations; data preprocessing and analysis; model building, fine-tuning and evaluation with PyTorch, TensorFlow and Scikit-Learn; REST API microservice integration; and Git and GitHub discipline with review before merging.

Each unit of work followed the same short cycle: discussion of the requirement, implementation on a branch, local testing, and review before merge. Ingestion, retrieval and generation were built and reviewed as separate steps, so that a problem could be traced to the stage that caused it, for example a poor answer caused by a poorly chosen passage rather than by the prompt.

4.11 RESULTS AND OUTPUTS

The project is currently under development, with ongoing implementation and testing of the document ingestion pipeline, retrieval quality and grounded answer generation, delivering retrieval over a document collection with cited, grounded answers.

At this stage the pipeline can take a document collection through cleaning, chunking, embedding and indexing, retrieve and rerank passages for a question, and return an answer that cites the passages it used. Retrieval quality was tuned by adjusting chunk size, the embedding model and the ranking step, with queries, answers and feedback logged so that quality could be reviewed and compared between changes.

4.12 SCREENSHOTS / DEMONSTRATION EVIDENCE

Demonstration evidence is provided through the architecture, query lifecycle and mind map figures of this report, tracing a single question from query to grounded answer and collecting the internship activities into a single view.

Figure 3: Mind Map of Internship Activities and Learning



Figure 3: Mind map of internship activities and learning

4.13 ADDITIONAL PROJECT FOR REFERENCE: LEARNLOCK

4.13.1 Overview

LearnLock is an Android-only Flutter application that blocks user-selected apps and unlocks them only after the user answers a multiple-choice question correctly. The questions are generated by an AI model behind a FastAPI backend and cached locally in SQLite, so the locking behaviour keeps working even when the device is offline. The project is included in this report as an additional project for reference; it shows how a mobile client, a native enforcement layer and a backend service are separated and connected in practice.

4.13.2 Problem Statement

Screen-time tools usually rely on a plain timer or a password, which a user can dismiss without effort. Study and focus applications need friction that is useful

rather than merely irritating. LearnLock turns the moment of distraction into a moment of revision: to open a blocked app the user must first demonstrate knowledge of a topic they chose, at a difficulty they chose.

4.13.3 How the Application Works

Onboarding. The user picks two to five topics, sets a difficulty for each topic (beginner, intermediate or advanced), chooses the apps to lock, sets the unlock timer, and grants the two required permissions: Usage Access and Display over other apps.

Enforcement. A Kotlin foreground service watches the active app through UsageStatsManager. When a locked app comes to the front, the service launches a native lock screen, LockActivity, which presents a quiz.

Answering. A correct answer gives a time-boxed unlock. A wrong answer allows one retry with a new question. After two failures the user may take an emergency five-minute unlock or quit. Every outcome is logged locally for on-device analytics.

Refill. When the pool of usable questions falls to between ten and fifteen, a silent batch is fetched in the background. A set of bundled fallback questions keeps locking functional when there is no internet connection.

4.13.4 System Architecture

The application is organised into three layers, summarised in Table 1 and Figure 4. The Flutter layer owns the interface, the local database and the question lifecycle. The Kotlin layer owns everything that must keep running outside the Flutter engine: foreground-app detection, the lock screen and the enforcement state. The FastAPI backend does one job only, question generation and validation, so that no AI key is ever shipped inside the APK.

Table 1: Layers of the LearnLock application

Layer	Stack	Responsibility
Flutter	Riverpod, go_router, sqflite, dio	Interface, onboarding, local database, analytics and question lifecycle
Kotlin	MethodChannel com.learnlock/native, MonitorService (foreground service), overlays, SharedPreferences	Foreground-app detection, lock screen and enforcement state
FastAPI	Python, OpenAI-compatible json_schema output, Cloud Run	Question generation and validation only; AI keys never ship in the APK

Figure 4: System Architecture of LearnLock

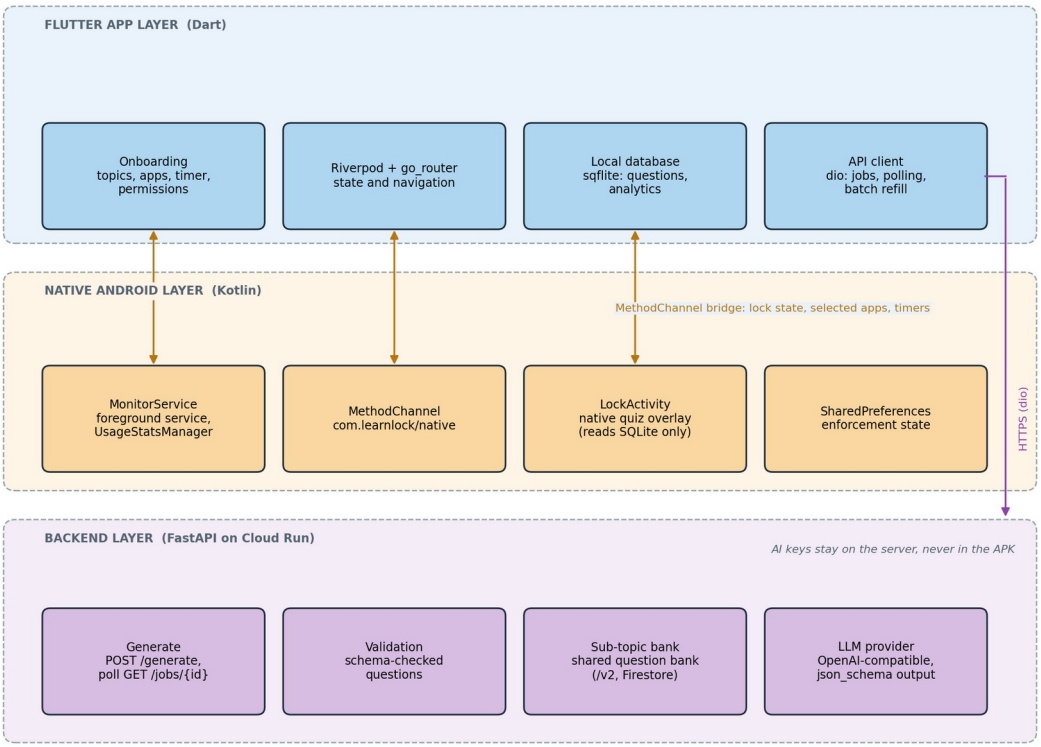


Figure 4: System architecture of LearnLock

Figure 5 follows a single lock event. The sequence is entirely local once a question pool exists: the lock screen reads questions from SQLite and records the result, and the network is touched only by the background refill.

Figure 5: LearnLock Lock, Answer and Unlock Lifecycle

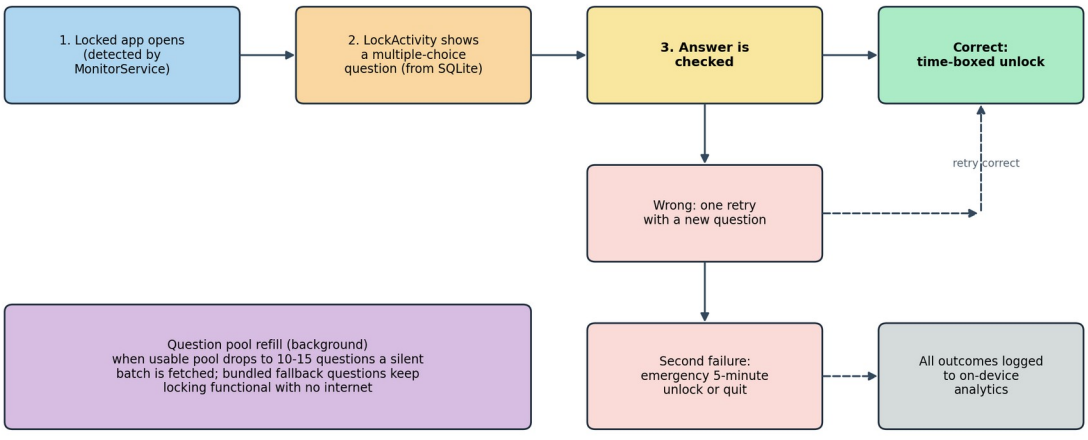


Figure 5: LearnLock lock, answer and unlock lifecycle

4.13.5 Key Design Principles

Three rules shaped the design. First, the lock screen reads SQLite only and never touches the network, so a slow or absent connection can never leave a user locked out or stuck on a blank screen. Second, analytics stay on the device. Third,

because generating twenty-five to fifty questions takes minutes, the version 1 backend interface uses asynchronous jobs: the client sends a generate request and then polls a job endpoint for the result, instead of holding a connection open.

4.13.6 Question Bank Backend (Ongoing Work)

The current focus of the project is turning question generation into a reusable, scalable sub-topic bank. The planned design is a shared, Firestore-backed bank organised by a taxonomy of twelve categories, about ninety-six nodes and roughly thirty sub-topics each, with fifty questions kept per sub-topic and topped up lazily on demand. New version 2 endpoints (bank ensure, bank next, jobs and bank export) sit beside the unchanged version 1 interface. A single-filler lease prevents duplicate AI spend when several devices request the same sub-topic, and an install identifier with a server-side record of questions already seen prevents repeats for a user. On the Flutter side, the bank is warmed up quietly during onboarding so that navigation stays instant, and answer choices are shuffled on the client with a seeded shuffle.

4.13.7 Relevance to the Internship

LearnLock is not part of the RAG chatbot but exercises the same habits: work divided into layers with narrow interfaces, a backend that wraps a language model behind validation, structured model output checked against a schema, and an offline-first client. It is documented here so that the reader can compare two different ways of placing a language model inside a product.

4.14 TESTING AND VALIDATION APPROACH

Testing followed the incremental rhythm of the internship. Each stage of the retrieval pipeline was tested on its own before the stages were joined: ingestion was checked for documents that failed to load or produced empty passages, chunking was checked so that passages kept enough context to be useful, and retrieval was checked by asking questions whose answers were known to be in the collection and confirming that the right passages came back near the top.

Only after retrieval behaved sensibly was generation judged. Answers were checked to confirm that they used the retrieved passages, that the citations pointed to the passages actually used, and that a question with no support in the collection did not receive a confident invented answer. Queries, answers and user feedback were logged so that two versions of the pipeline could be compared on the same questions.

A short checklist was used before each merge: the change does one thing, its tests pass, naming follows the existing code, and the commit message explains the reason for the change.

CHAPTER 5
SKILLS AND LEARNING OUTCOMES

5.1 TECHNICAL SKILLS

A working understanding of retrieval-augmented generation end to end: ingestion, embeddings, retrieval, prompt assembly and grounded generation. Hands-on experience building, fine-tuning and evaluating models with PyTorch, TensorFlow and Scikit-Learn.

5.2 SOFTWARE / TOOLS / PLATFORMS

Python, PyTorch, TensorFlow, Scikit-Learn, REST API microservices, and Git and GitHub with review before merge and testing.

5.3 AI / CLOUD / DATA SKILLS

Data cleaning, preprocessing, exploratory analysis, feature engineering and hyperparameter optimisation applied to both tabular data and documents, including document loading, cleaning and passage splitting for embeddings and retrieval quality.

The practice of evaluating retrieval separately from generation was the most valuable lesson from the data side of the internship, since it shows which stage is responsible when an answer is poor.

5.4 COMMUNICATION AND TEAMWORK

Improved technical communication through interaction with Karthikeyan, Software Developer, for requirement understanding, technical clarification, code-related guidance and project feedback.

5.5 PROBLEM-SOLVING SKILLS

Iterating chunk size, embeddings and ranking for retrieval quality; assembling prompts so the model answers from retrieved context with citations; and preparing inconsistent real source files.

5.6 PROFESSIONAL LEARNING

Professional working habits: version control on Git and GitHub, review before merge, and testing, with an appreciation of layered architecture as a means of containing change.

The internship also built confidence in working inside an unfamiliar codebase: reading before writing, following established conventions, and asking for review early instead of late.

CHAPTER 6
CHALLENGES AND SOLUTIONS

6.1 TECHNICAL CHALLENGES

Retrieval quality: an answer is only as good as the passages retrieved, and chunk size, embeddings and ranking took more iteration than generation. Grounding and citations: keeping the model on retrieved context rather than memory required careful prompt assembly and checking. Document preparation: inconsistent real source files needed heavy cleaning and chunking. Evaluation: fluent answers can still be wrong or ungrounded. Working inside an existing codebase with established conventions.

Scope was a further challenge. A retrieval-augmented system touches document handling, embeddings, a vector store, a language model and an interface, and it was not always obvious which stage a given problem belonged to. Understanding the stages and their interfaces was necessary before even a small change could be made with confidence.

6.2 ORGANIZATIONAL / TEAM CHALLENGES

Contributing to code written by others required reading and following existing conventions, delivering in reviewable increments rather than one late submission.

There was also the discipline of time. Units of work were short, and a task that was not broken down well at the start could run past its planned time. Learning to estimate small tasks realistically, and to say early when one was growing, was part of the professional growth that the internship provided.

6.3 SOLUTIONS ADOPTED

Structured preprocessing and repeatable ingestion with an updatable index; incremental implementation with version control and review before merge; retrieval and answer quality testing; and mentor guidance on requirements and implementation quality.

6.4 KEY LESSONS LEARNED

Treat a project as an end-to-end system and deliver in reviewable increments, grounding every answer in retrieved sources so it can be traced.

A further lesson was that each stage of a retrieval pipeline needs its own checks. An answer can only be as good as the passages retrieved, so retrieval was evaluated on its own before generation was judged. Keeping changes small and reviewable made it possible to see which change had improved or worsened the results.

CHAPTER 7
PROFESSIONAL EXPERIENCE

7.1 WORKPLACE PRACTICES

Incremental work with requirement discussion, implementation, review and integration; version control on Git and GitHub; and review before merge kept partially finished work out of the product.

7.2 INDUSTRY STANDARDS

Production-ready code artifacts, validation at service boundaries, error handling, and predictable behaviour under repeated calls.

Industry practice also expects that a service describes its own limits: what it accepts, what it rejects and why. Following these standards in the chatbot's interfaces meant that clients of the service could rely on consistent behaviour, including a clear response when no supporting passage was found.

7.3 PROFESSIONAL ETHICS

Accurate record-keeping of internship activities as advised at the review meeting, with honest evaluation of models and work on evidence.

Professional ethics also covered respect for the work of others. Existing code was extended in the style in which it was written, review comments were treated as guidance rather than criticism, and the contributions of the mentor and the team are acknowledged in this report.

7.4 SAFETY, SECURITY AND DATA PRACTICES

Strict confidentiality regarding proprietary source code, algorithms, model weights, studio datasets and internal configuration belonging to Pyntree Technologies. This report describes system structure, data flow and design reasoning only, with no source listings, weights, credentials, dataset samples or internal configuration reproduced.

Data practices followed the same principle. Documents were used only for the purpose of the project, were handled within the company's environment, and are not reproduced in this report in any identifiable form.

The internship agreement also requires that proprietary material is not shared outside the company. For that reason, figures in this report show architecture and workflow at a conceptual level, and the tables describe layers and responsibilities by purpose rather than by implementation detail.

CHAPTER 8
CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION

The 2 months internship at Pyntree Technologies met its objective of moving from academic familiarity with AI and machine learning to practical participation in production software development. Working on the RAG chatbot placed the machine learning work in a context where it had to serve a real workflow: a language model is useful over an organisation's material only when retrieval feeds it the right context, and an answer is trustworthy only when it can be traced to its sources.

Equally important was the change in perspective. At the start of the internship, success was measured by whether a model trained and produced a good score. By the end, success was measured by whether an answer was correct, could be traced to its sources, survived a review, and could be maintained by someone else.

8.2 ACHIEVEMENT OF OBJECTIVES

Hands-on model building, fine-tuning and evaluation, data preprocessing and analysis, Python framework use, REST API integration and version control were achieved through the RAG chatbot.

Each objective stated in Chapter 1 was met in a form that can be pointed to: a repeatable ingestion and indexing path, a retrieval step whose quality was tuned and compared, grounded answers with citations, REST API integration, and a history of reviewed commits.

8.3 CAREER RELEVANCE

The internship established end-to-end RAG understanding and production habits that carry forward into the final-year project and professional AI engineering work.

8.4 FUTURE SCOPE / RECOMMENDATIONS

The project continues under development, with ongoing implementation and testing of the document ingestion pipeline, retrieval quality and grounded answer generation.

Recommendations for continuing the work include extending automated tests across the retrieval pipeline, measuring answer quality on a fixed set of questions after every change, and evaluating larger document collections.

REFERENCES

IV-Year Project First Review Meeting Report, Department of Artificial Intelligence and Data Science, review meeting dated 17.08.2026, Academic Year 2026-2027.

Project documentation and development records, RAG chatbot, Pyntree Technologies (internal, not reproduced).

Technical guidance and code review notes from the industry mentor, Karthikeyan, Software Developer.

Python machine learning frameworks used during the internship: PyTorch, TensorFlow and Scikit-Learn.

Lewis, P. et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, Advances in Neural Information Processing Systems (NeurIPS), 2020.

Vaswani, A. et al., Attention Is All You Need, Advances in Neural Information Processing Systems (NeurIPS), 2017.

Pedregosa, F. et al., Scikit-learn: Machine Learning in Python, Journal of Machine Learning Research, vol. 12, 2011.

Paszke, A. et al., PyTorch: An Imperative Style, High-Performance Deep Learning Library, Advances in Neural Information Processing Systems (NeurIPS), 2019.

Abadi, M. et al., TensorFlow: A System for Large-Scale Machine Learning, USENIX OSDI, 2016.

Chacon, S. and Straub, B., Pro Git, Apress (available at git-scm.com).

Sommerville, I., Software Engineering, Pearson Education.

APPENDICES

Appendix A - Internship Certificate

This is to certify that Renuka S (Register Number 922123243043) completed an internship in AI/ML at Pyntree Technologies from 01 July 2026 to 01 September 2026, working on RAG Chatbot - An AI Knowledge Assistant under Karthikeyan, Software Developer. Company address: 1st Floor, SMA Complex, Mendonsa Colony, Nagal Nagar, Dindigul, Tamil Nadu - 624003. The completion certificate is reproduced in full on page 30, at the end of the report.

Appendix B - Internship Offer / Permission Letter

Internship engagement in AI/ML at Pyntree Technologies from 01 July 2026 to 01 September 2026 as AI/ML Intern, covering AI/ML model development, data preprocessing and analysis, model evaluation, and REST API integration with Python, PyTorch, TensorFlow, Scikit-Learn and Git/GitHub, under developer mentorship with review meeting dated 17.08.2026. A copy of the offer letter is reproduced in full on page 3, after the Bonafide Certificate.

Appendix C - Weekly Internship Logbook

Month 1: foundations, data and models - product and codebase understanding, environment and Git/GitHub setup, cleaning, preprocessing and exploratory analysis, building, fine-tuning and evaluation with PyTorch, TensorFlow and Scikit-Learn. Month 2: retrieval, generation and integration - chunking, embeddings and retrieval, grounded generation with citations, REST API integration, testing and quality improvement.

Table 2: Weekly internship logbook

Week	Period	Activity
1	01 Jul - 07 Jul 2026	Orientation to the product and codebase; development environment and Git/GitHub setup; data collection and cleaning.
2	08 Jul - 14 Jul 2026	Data preprocessing, exploratory analysis and feature engineering; baseline models with Scikit-Learn.
3	15 Jul - 21 Jul 2026	Model building and training with PyTorch and TensorFlow.
4	22 Jul - 28 Jul 2026	Fine-tuning, hyperparameter optimisation and evaluation on held-out data.
5	29 Jul - 04 Aug 2026	Document loading, cleaning and chunking for the RAG pipeline.
6	05 Aug - 11 Aug 2026	Embedding of passages and queries; indexing into the vector store.
7	12 Aug - 18 Aug 2026	Semantic retrieval with top-k search and reranking; prompt assembly.
8	19 Aug - 25 Aug 2026	Grounded answer generation with citations; REST API integration with the chat orchestrator.
9	26 Aug - 01 Sep 2026	Testing, retrieval-quality improvement, feedback and logging, and final handover of work.

Appendix D - Industry Mentor Evaluation

Industry mentor Karthikeyan, Software Developer, guided requirement understanding, technical problem approaches and implementation quality, with interaction for technical clarification, code-related guidance and project feedback.

Table 3: Areas of guidance from the industry mentor

Area	Guidance received
Understanding of requirements	Guided the student in reading the product requirements and breaking them into reviewable units of work.
Technical problem approaches	Advised on approaches to retrieval quality, chunking, embeddings and ranking, and on structuring model experiments.
Implementation quality	Reviewed code before merge and gave feedback on structure, readability and error handling.
Technical clarification	Available for questions and clarification throughout the internship.
Code-related guidance	Guidance on Git and GitHub practice, branching, commits and review before merging.
Project feedback	Regular feedback on progress of the RAG chatbot work and on the quality of the outputs.

Mentor: Karthikeyan, Software Developer, Pyntree Technologies.

Appendix E - Final Internship Presentation / Viva Record

The review meeting collected and verified the internship details, AI/ML training, software-development responsibilities, project involvement and practical learning. The student was advised to maintain proper records and continue developing the RAG chatbot based on the technical experience gained.

Appendix F - Photographs, Screenshots and Project Evidence

Project evidence figures are included in Chapter 4 of this report (architecture, workflow/lifecycle and mind map), sourced from the internship project record.

Figure 1: System Architecture of the RAG Chatbot

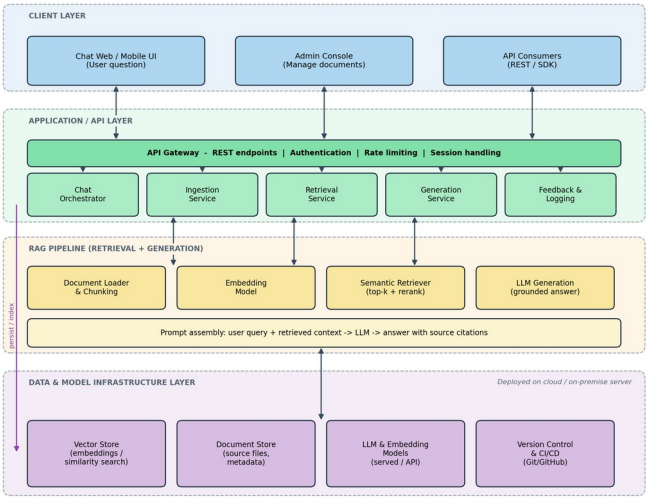


Figure 1: System architecture of the RAG chatbot

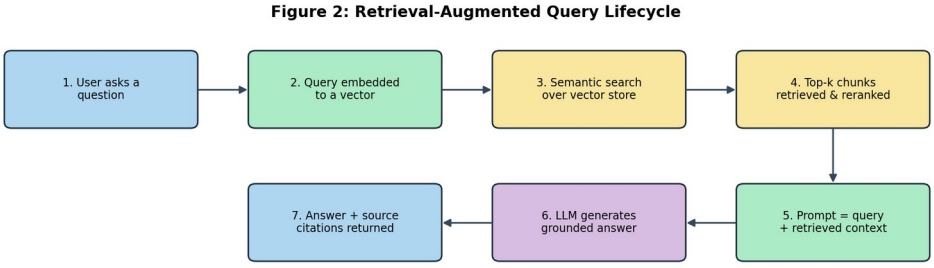


Figure 2: Retrieval-augmented query lifecycle

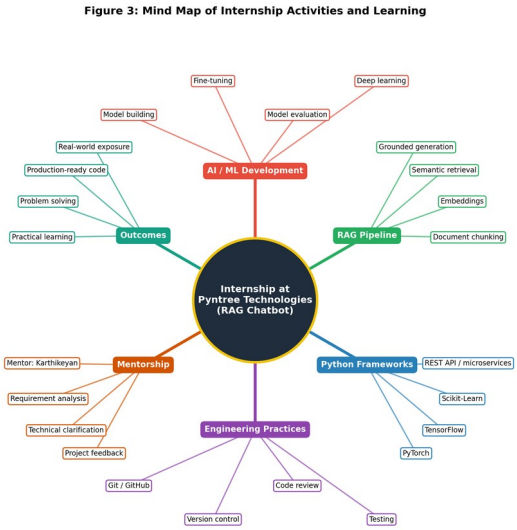


Figure 3: Mind map of internship activities and learning

COMPLETION CERTIFICATE

INTERNSHIP CERTIFICATE

Pyntree technologies

This is to certify that **Renuka S** from **SSM Institute of Engineering and Technology** successfully completed her internship of 2 months from **01 July 2026** to **01 Sep 2026** at Pyntree Technologies.

Internship: AI RAG Deveopment

During her tenure, she learned **AI RAG Development** for our client website chatbot. We found her active and competent in executing all assigned tasks and services were found to be satisfactory.

We wish her great success in all her future endeavours.

Date : Sep 02, 2026

Place : Dindigul, Tamilnadu.

Sincerely,

Karthik Babulal

Founder and Director

Pyntree Technologies

PYNTREE TECHNOLOGIES
133, SMA Complex, RS Road, Dindigul – 624003
Email: hello@pyntree.com
CERT ID: 2026/1002